CereForge

Forge Your Mind.

Open-Source Competitive Learning Platform for AI & Autonomous Systems Engineers

Author / Lead Engineer: Siddharth Patni

Repository: github.com/Siddharthpatni/Cereforge

Date: February 2026

Table of Contents

Author's Note

1. **Executive Summary**
2. **Problem-Solution Mapping**
   - 2.1 The Tutorial Hell Trap
   - 2.2 Missing Production Engineering Context
   - 2.3 No Domain-Specific Community Infrastructure
3. **Technical Architecture**
   - 3.1 Stack Overview
   - 3.2 Key Decisions and the Alternatives Rejected
   - 3.3 What I Got Wrong and Had to Redo
4. **Platform Features**
   - 4.1 Challenge System
   - 4.2 Gamification
   - 4.3 Community Q&A
   - 4.4 Adaptive Learning Paths
5. **Security & Quality Assurance**
   - 5.1 Authentication & Authorisation
   - 5.2 Testing Strategy
6. **Infrastructure & Deployment**
7. **On Automated Grading -- The Question I Expect**
   - 7.1 Why Self-Reporting is Pedagogically Defensible
   - 7.2 Why Automated Grading for These Problems is Hard
   - 7.3 The Planned Implementation
8. **Research Directions & Future Scope**
9. **API Reference**
10. **Contributing & License**

Author's Note

I spent the better part of a year doing exactly what CereForge is designed to stop. ML tutorials. ROS2 simulations. Disease prediction notebooks. Flask apps I built and then forgot about because they didn't connect to anything real. I had CGPA scores and certifications and absolutely nothing I could point to and say: I shipped that, it runs, people use it.

The gap isn't knowledge. Every serious learner I've spoken to has plenty of that. The gap is the jump from understanding a concept to making an architectural decision under constraints -- latency, cost, data quality, maintainability. That jump is what gets you hired, and no tutorial I found actually makes you take it.

So I built the platform I needed. CereForge is the answer to a problem I lived with.

> *One honest note upfront: automated solution evaluation is not yet built. Tasks are self-reported. I know this is the first question a serious reviewer will ask, and I address it directly in Section 7 -- including why self-reporting is a deliberate pedagogical choice and why automated grading for open-ended architectural decisions is a harder research problem than it first appears.*

1. Executive Summary

CereForge is a production-grade, open-source learning platform targeting the gap between theoretical AI education and applied engineering practice. It is not a course catalog, a video library, or a quiz platform. It is a system that forces engineers to make real decisions -- about architecture, cost, latency, and tradeoffs -- in 12 structured challenges across 4 production AI domains.

Built on a fully async Python/React stack with JWT auth, background task processing, streaming AI responses, and a containerized local development environment, CereForge is itself a demonstration of the skills it teaches.

12 Challenges	4 Learning Tracks	5 Rank Tiers	3 Structured Paths
Beginner to Expert per track	LLM - RAG - CV - Agents	Trainee to Elite	30 / 21 / 28 days

2. Problem-Solution Mapping

Each feature in CereForge maps directly to a structural failure I observed in my own learning and in conversations with other engineers at the same stage. This section documents those failures and the specific design responses.

2.1 The Tutorial Hell Trap

The Problem. Most online AI courses train people to run code, not to make decisions. You learn that transformers use self-attention, but you never have to choose between fine-tuning a base model and prompt-engineering a frontier one given a $400/month inference budget and a 200ms P95 latency requirement. The decision-making muscle never gets exercised.

The CereForge Response. Every challenge simulates a real production constraint. The LLM Engineering beginner task is not 'implement attention' -- it is 'design a three-step prompt chain for a product review classifier that must handle sarcasm and stay under 1,500 tokens per call.' The framing forces a tradeoff, not a recitation.

> *I built the first three tasks directly from situations I had gotten stuck on or seen discussed on forums with no good answers. The specificity isn't padding -- it's the point.*

2.2 Missing Production Engineering Context

The Problem. AI courses exist in isolation from software engineering. Learners who can fine-tune a model don't know what a background task queue is for, why you'd stream an LLM response rather than await it, or what happens to your API when three users hit the AI Mentor endpoint simultaneously on a synchronous server.

The CereForge Response. The platform itself is the lesson. The codebase uses async SQLAlchemy, offloads badge computation to Celery workers, and streams AI Mentor tokens via SSE. A user who reads the open-source code alongside completing the challenges sees these patterns implemented and explained in context -- not as abstract best practices, but as solutions to specific problems the platform itself faces.

2.3 No Domain-Specific Community Infrastructure

The Problem. General programming forums are structurally hostile to AI engineering questions. Questions about vector store chunking strategies or LangGraph agent memory get buried, attract one-line responses, or get closed as 'too broad.' The knowledge exists, but the infrastructure to surface it doesn't.

The CereForge Response. A Stack Overflow-style Q&A engine built specifically for AI engineering, with voting, accepted answers, and track-based filtering. XP awarded for accepted answers creates a direct economic incentive for quality -- the same mechanism that made Stack Overflow work -- applied to a domain that needs it.

3. Technical Architecture

3.1 Stack Overview

Layer	Technologies	Version / Notes
Frontend	React, Vite, Tailwind CSS, Framer Motion	React 18, Vite 5
State / Data	Zustand (UI state), React Query (server state)	Strict separation of concerns
Rich Text	TipTap editor	Markdown + code blocks
Backend / API	FastAPI, SQLAlchemy 2.0 async, Pydantic v2	Python 3.11, fully async
Auth	JWT (access + refresh), Bcrypt	15-min access tokens, rotated refresh
Task Queue	Celery + Redis	Badge compute, XP agg, email dispatch
Database	PostgreSQL via SQLAlchemy async	Alembic version-controlled migrations
AI Integration	Anthropic Claude API	Streamed via SSE, skill-adaptive prompts
Infra / Deploy	Docker Compose, GitHub Actions, Vercel, Render	Full prod parity in local dev

3.2 Key Decisions and the Alternatives Rejected

This section documents not just what was chosen, but what was considered and why it lost. A list of technologies is easy to generate. The reasoning is harder -- and more useful.

FastAPI over Django or Flask

Django's ORM is synchronous by default. On a platform where users hit the AI Mentor endpoint concurrently -- each request blocking for 2-8 seconds on a Claude API call -- a synchronous server would serialize those requests or require a thread-per-request model that doesn't scale cheaply. FastAPI with asyncio allows hundreds of concurrent in-flight LLM requests on a single worker process. Flask was not a serious contender for the same reason: no native async support.

> *The honest tradeoff: FastAPI's smaller ecosystem meant writing more boilerplate for things Django gives for free -- admin interface, form handling. For this platform, that was the right trade.*

Celery + Redis over a Managed Queue

Badge computation, XP aggregation, and email dispatch happen after a task submission response has already returned to the user. Blocking on these in the request lifecycle would add 200-800ms of perceived latency for no user-visible reason. Celery handles this cleanly. SQS and Google Cloud Tasks were considered for managed reliability, but both add infrastructure cost and vendor lock-in not justified at current scale. Redis doubles as both the Celery broker and a cache layer -- one service, two jobs.

Zustand plus React Query over Redux

Redux treats all state -- local UI toggles and server-fetched data -- as the same kind of thing. They aren't. Server state has staleness, cache invalidation, background refetching, and optimistic update semantics that Redux requires significant boilerplate to handle. React Query manages server state correctly by default. Zustand handles the remaining UI state with almost no configuration. The two libraries compose cleanly and the separation makes the codebase easier to reason about.

Streaming SSE for AI Mentor Responses

The alternative is a standard await-and-respond: wait for the full Claude response, then send it. At typical LLM generation speeds (40-80 tokens/second for a 300-token response), that means 3-7 seconds of blank UI. Server-Sent Events stream tokens as they arrive. The user sees the first token in under 500ms. Perceived performance is dramatically better even though total response time is identical.

3.3 What I Got Wrong and Had to Redo

The initial database schema modelled XP as a single integer field on the User table, incremented on task completion. This broke immediately when community Q&A went in: users could earn XP from accepted answers and votes, and I needed a full audit trail for the leaderboard. I had to migrate to an XP transaction log table and rewrite the aggregation logic. The migration itself was clean -- Alembic handles it -- but the schema redesign cost about two days.

> *The lesson wasn't 'design your schema correctly the first time' -- I'm not sure that's possible for a system you're designing while building. The lesson was that Alembic makes this kind of change tractable. Version-controlled migrations aren't just good practice; they're what make iterative schema design survivable.*

4. Platform Features

4.1 Challenge System -- 12 Tasks Across 4 Tracks

Track	Core Engineering Decisions Practised	Count	XP Range
LLM Engineering	Prompt chain design, token budget optimisation, fine-tuning strategy	3	50-300
RAG Pipelines	Vector store selection, chunking strategy, hallucination detection	3	50-300
Computer Vision	Object detection tradeoffs, multimodal design, edge deployment	3	50-300
Autonomous Agents	LangGraph state machines, multi-agent orchestration, failure handling	3	50-300

Every challenge ships with: a plain-English beginner guide, a progressive hint revealed on demand, curated external resources, a Google Colab notebook for practical implementation, and AI Mentor access.

4.2 Gamification

The XP and badge system exists for a specific reason: completion rates on self-directed learning platforms are abysmal (MOOCs average under 10%). Visible progress, ranked standing, and unlock animations are the mechanism by which the platform creates extrinsic motivation to push through the first uncomfortable hours of a new domain.

- **XP System:** earned for challenge completion, community answers, and accepted answer bonus. Tracked via append-only transaction log, aggregated for leaderboard.
- **5 Rank Tiers:** Trainee to Engineer to Architect to Autonomous to CereForge Elite. Each tier has a defined XP threshold -- a user's tier is stable, not relative.
- **12 Badges:** domain-specific achievements with Framer Motion unlock animations. Cinematic feedback at the moment of achievement is not gratuitous -- it is the dopamine hit that makes the next challenge feel worth starting.
- **Live Leaderboard:** XP aggregation triggered via Celery on every XP event, ensuring real-time standings without blocking user-facing requests.

4.3 Community Q&A

Stack Overflow-style forum purpose-built for AI engineering. The filtering system -- by track, difficulty, tag, and a beginner-friendly toggle -- was the feature I most underestimated. Getting the SQL queries right for multi-dimensional filtering with correct vote aggregation took longer than building the entire voting system itself.

4.4 Adaptive Learning Paths

Path	Duration	Entry Requirement	Task Count
Zero to AI -- Complete Beginner	30 days	No experience assumed	6 tasks
Production RAG Systems	21 days	Intermediate -- Python + APIs	5 tasks
Autonomous AI Agents Masterclass	28 days	Advanced -- prior LLM experience	4 tasks

5. Security & Quality Assurance

5.1 Authentication & Authorisation

- **JWT access tokens:** 15-minute expiry. Short lifetime limits the blast radius of a leaked token without requiring database lookups on every request.
- **Refresh token rotation:** refresh tokens are single-use and rotated on each use. A stolen refresh token can only be used once before invalidation.
- **Bcrypt with salting:** plaintext passwords are never stored, never logged, never appear in error messages. Salt rounds set to 12.
- **Pydantic v2 validation at all API boundaries:** malformed inputs are rejected before they touch business logic or the ORM. Strict type coercion prevents bugs where '1' and 1 are treated as equivalent.

5.2 Testing Strategy

Backend coverage targeting >= 75%, with route handlers, service logic, and ORM operations as primary targets. Frontend: Vitest unit tests for component logic and Zustand store behaviour.

CI/CD: GitHub Actions runs the full test suite on every pull request. Merges are blocked if coverage drops or any test fails. Alembic migration checks run in CI to catch schema drift before it reaches staging.

6. Infrastructure & Deployment

Component	Service	Rationale
Frontend	Vercel	Global edge CDN, zero-config React deploys, instant rollback
Backend API	Render	Automatic PR preview deployments; no Kubernetes overhead at current scale
Database	Supabase (PostgreSQL)	Managed backups, connection pooling via PgBouncer, generous free tier
Task Queue	Redis + Celery	Self-hosted on Render; doubles as cache layer
Local Dev	Docker Compose	Full stack in one command; identical to production
CI/CD	GitHub Actions	Test on every PR, block merge on failure, deploy on merge to main

The Docker Compose setup is not just for convenience -- it is the mechanism that guarantees environment parity. 'Works on my machine' is not an acceptable failure mode for a platform whose explicit purpose is teaching production engineering.

7. On Automated Grading -- The Question I Expect

Task completion is currently self-reported. Users submit a solution, mark it complete, and earn XP. There is no automated evaluation. I want to address this directly because it is the most obvious gap in the platform, and the answer is more nuanced than it might appear.

> **Known limitation:** Automated solution evaluation is not yet implemented. Self-reporting is the current mechanism. This section explains why this is a deliberate interim choice and what automated evaluation actually requires to do correctly.

7.1 Why Self-Reporting is Pedagogically Defensible

The challenges on CereForge are not competitive programming problems. There is no single correct answer to 'design a RAG pipeline for a legal document search system with a $200/month budget and a 95% recall requirement.' The solution space is large and the evaluation criteria are multidimensional: cost, latency, accuracy, maintainability, explainability.

A unit-test-based grader would reduce these problems to the subset of criteria that can be expressed as assertions. That would change the nature of the challenges. It would reward learners who reverse-engineer the test suite rather than those who reason well about tradeoffs.

> *This is the same reason case interviews at McKinsey don't have answer keys. The reasoning process is the thing being evaluated, not the output.*

7.2 Why Automated Grading for These Problems is Hard

The correct approach for open-ended architectural evaluation is LLM-as-judge: a structured prompt presenting the problem, the evaluation rubric, and the submitted solution, asking a capable model to score across defined dimensions. This is an active research area. The core challenges are:

- **Scoring consistency:** LLM judges exhibit variance across identical submissions. RAGAS (Shahul et al., 2023) and MT-Bench (Zheng et al., 2023) show that judge consistency requires careful rubric design and temperature control.
- **Rubric calibration:** without ground-truth scores from human expert raters on a calibration set, it is not possible to validate that the judge's scoring aligns with what a senior engineer would award.
- **Gaming resistance:** once learners know an LLM is evaluating their submission, there is an incentive to write fluent persuasive text rather than technically sound architecture.

Building this correctly is a research project in its own right. It is listed in Section 8 as the primary future development priority.

7.3 The Planned Implementation

Target architecture: a sandboxed evaluation endpoint that accepts a structured solution submission (architecture rationale and optionally a code artifact), passes it through a multi-dimensional rubric prompt to Claude, and returns a score with dimension-level feedback.

Rollout plan: build on three expert-tier challenges first, calibrate against scores from two independent human raters, and expand only once judge consistency reaches acceptable thresholds (targeting Cohen's kappa >= 0.7 against human raters).

8. Research Directions & Future Scope

The following are not wishlist items. Each maps to a specific current limitation of the platform and has a defined research question.

8.1 Automated Solution Evaluation (Primary Priority)

As described in Section 7. Research question: can LLM-as-judge achieve Cohen's kappa >= 0.7 against senior-engineer raters on open-ended AI architecture problems, and what rubric design choices drive consistency? Relevant prior work: RAGAS (Shahul et al., 2023), MT-Bench (Zheng et al., 2023).

8.2 RAG Hallucination Benchmarking

Current RAG challenges evaluate learner-built pipelines through self-assessment. The goal is to integrate standardized benchmarks (MMLU, HELM, TruthfulQA) so submitted pipelines are evaluated against research-grade test sets, producing a measurable hallucination rate rather than a subjective quality score.

8.3 Edge AI Deployment Constraints

The Computer Vision track currently presents edge deployment as a design constraint in prose. Real telemetry -- measured power budgets, inference latency profiles on ARM Cortex-M and Jetson Nano class hardware -- would make these challenges genuinely harder and genuinely more useful. This is a hardware-lab collaboration problem, not a software problem.

8.4 Multi-Agent Safety Diagnostics

Multi-agent systems exhibit failure modes single-agent systems don't: cascading hallucinations, instruction drift across agent hops, deadlock in circular delegation chains. Developing diagnostic challenges based on documented failure taxonomies from recent LangGraph and AutoGen research would make the Autonomous Agents track significantly more rigorous.

9. API Reference

Full interactive documentation via Swagger UI at http://localhost:8000/docs. All endpoints use JSON and require JWT Bearer token authentication except /auth/register and /auth/login.

Method	Endpoint	Description
POST	/api/v1/auth/register	Create account; skill level personalises AI Mentor prompts
POST	/api/v1/auth/login	Returns short-lived access token + rotating refresh token
GET	/api/v1/auth/me	Current user profile, rank tier, badge inventory
GET	/api/v1/tasks	List challenges; filter: track, difficulty, completion status
POST	/api/v1/tasks/{slug}/submit	Record completion; triggers async Celery job for XP + badge evaluation
GET	/api/v1/posts	Community Q&A; filter: track, tag, difficulty, beginner-friendly
POST	/api/v1/posts	Create question; triggers tag extraction and track classification
POST	/api/v1/vote	Vote on post or answer; idempotent; score aggregated at read time
GET	/api/v1/leaderboard	Live global rankings; XP aggregated from transaction log via Celery
GET	/api/v1/dashboard	Personalised dashboard: recommended next task, completions, badges
POST	/api/v1/ai-mentor/guidance	Streamed SSE response; prompt varies by user skill level and challenge context
GET	/api/v1/health	Health check; returns DB and Redis connectivity status

10. Contributing & License

10.1 Contributions

The highest-value contribution is new challenge content. If you have solved a real AI engineering problem in production -- a RAG pipeline that failed in an interesting way, an agent architecture that required an unexpected tradeoff -- and can frame it as a well-specified challenge, open a GitHub issue. That knowledge is exactly what the platform is designed to preserve and transmit.

Code contributions: open an issue before building anything substantial, fork, branch, PR. All tests must pass and coverage must not drop. One PR per feature or fix.

10.2 License

MIT. See LICENSE in the repository root. Free to use, fork, and extend for educational and commercial purposes.

Built by an engineer who got tired of tutorials that stop before production.

github.com/Siddharthpatni/Cereforge